

Acoplamento e coesão



acoplamento

No desenvolvimento de sistemas, dois conceitos essenciais no design de software são o acoplamento e a coesão. Esses conceitos impactam diretamente a estrutura e a manutenção do sistema, influenciando sua flexibilidade e organização. Nesta primeira parte, vamos entender o que são acoplamento e coesão e como eles se relacionam na prática.

O que é Acoplamento?

O acoplamento refere-se ao grau de dependência entre os diferentes componentes ou classes de um sistema. Quando dois ou mais componentes estão fortemente acoplados, significa que um depende diretamente do outro para funcionar corretamente. Esse tipo de relacionamento pode ser prejudicial ao sistema, pois uma mudança em um componente pode forçar alterações em outros, o que aumenta a complexidade de manutenção.

Por outro lado, buscamos um baixo acoplamento, ou seja, componentes que interagem de maneira mínima e independente. Isso torna o sistema mais flexível e fácil de modificar, pois as mudanças em um componente têm menos impacto nos demais. No design de software, diagramas UML são frequentemente utilizados para identificar e minimizar o acoplamento entre classes e componentes, facilitando a visualização dessas interdependências.

O que é Coesão?

A coesão, por sua vez, refere-se ao quão bem as responsabilidades de um módulo ou classe estão agrupadas. Um módulo é considerado coeso quando todas as suas funcionalidades estão diretamente relacionadas e colaboram para alcançar um único objetivo. Alta coesão é uma característica desejável, pois simplifica o entendimento do código e facilita a sua manutenção.

Quando a coesão é baixa, um componente tende a assumir responsabilidades diversas e não relacionadas, o que gera confusão e dificulta tanto o uso quanto a manutenção. Em outras palavras, quanto mais coeso for um componente, mais claro e eficaz ele será em cumprir suas funções específicas.

Relação entre Acoplamento e Coesão

Um bom design de software busca manter o acoplamento baixo e a coesão alta. Esses dois conceitos são interdependentes: à medida que aumentamos a coesão num módulo, geralmente conseguimos diminuir o acoplamento com outros módulos. Isso ocorre porque módulos altamente coesos têm responsabilidades bem definidas e podem funcionar de maneira mais independente.

Por exemplo, se um componente possui várias responsabilidades desconexas (baixa coesão), ele pode acabar interagindo intensamente com outros componentes, aumentando o acoplamento. Já em um design com alta coesão, os módulos são projetados para fazer apenas o que lhes compete, minimizando a necessidade de dependências excessivas.

Por que isso é importante?

Manter um baixo acoplamento e uma alta coesão facilita a manutenção do sistema ao longo do tempo. Sistemas com essas características são mais fáceis de entender, testar, modificar e expandir, pois suas partes são independentes e bem organizadas. Isso torna o código mais robusto e menos suscetível a falhas em cascata, onde mudanças em um componente impactam várias outras partes do sistema.

Coesão

No design de software, a coesão é um conceito central que se refere ao nível de foco e organização interna de um módulo ou classe. Quanto mais relacionados e focados forem os elementos dentro de um módulo, maior será sua coesão. Nesta parte, vamos explorar o que é coesão e como aplicá-la no desenvolvimento de software.

O que é Coesão?

Coesão é a medida de quão bem os métodos e atributos de uma classe trabalham juntos para cumprir uma única responsabilidade. Uma classe com alta coesão possui métodos e atributos que estão claramente relacionados e trabalham em conjunto para um objetivo comum. Isso facilita a manutenção, entendimento e reutilização do código. Em contrapartida, uma classe com baixa coesão costuma lidar com muitas responsabilidades distintas, o que dificulta sua organização e clareza.

No contexto dos diagramas UML, a coesão pode ser observada pela lógica e agrupamento dos elementos em uma classe ou módulo. Se os atributos e métodos de uma classe são fortemente interligados e colaboram para uma mesma funcionalidade, essa classe tem alta coesão, que é o ideal em design de software.

Princípios que Promovem a Coesão

Existem alguns princípios de design de software que ajudam a manter a coesão alta. Três deles são especialmente importantes:

Princípio da Responsabilidade Única (SRP - Single Responsibility Principle): Este princípio estabelece que uma classe deve ter apenas uma razão para existir, ou seja, uma única responsabilidade. Se uma classe possui mais de uma responsabilidade, deve ser dividida para garantir que cada nova classe tenha apenas uma função clara, promovendo uma coesão maior.

Princípio Aberto-Fechado (OCP - Open/Closed Principle): Este princípio afirma que uma classe deve estar aberta para extensão e fechada para modificação. Isso significa que novas funcionalidades podem ser adicionadas através da criação de subclasses, sem que seja necessário modificar o código existente. Ao respeitar esse princípio, evitamos que uma classe acumule responsabilidades, preservando sua coesão.

Princípio da Segregação de Interfaces (ISP - Interface Segregation Principle): Este princípio recomenda que interfaces devem ser específicas e voltadas para suas funcionalidades. Isso significa que uma

classe não deve ser forçada a implementar métodos que não irá utilizar. Ao dividir interfaces maiores em interfaces menores e mais específicas, mantemos as classes coesas, focadas apenas nos métodos que realmente fazem parte de suas responsabilidades.

Exemplo Prático de Refatoração

Vamos considerar a classe GerenciadorHotel, que possui os métodos fazerReserva(), calcularConta() e enviarConfirmação(). Aqui, observamos que a classe está assumindo várias responsabilidades ao mesmo tempo, o que resulta em baixa coesão. Para corrigir isso, podemos aplicar uma refatoração que divide essa classe em três: GerenciadorReservas para lidar com as reservas, CalculadoraContas para tratar dos cálculos das contas, e EnviadorConfirmacoes para o envio de confirmações. Dessa maneira, cada nova classe possui uma única responsabilidade bem definida, aumentando a coesão.

Outro exemplo pode ser observado no uso de interfaces. Imagine uma interface de pagamento que processa diversos tipos de pagamentos, como cartão de crédito, boleto e Pix. Para seguir o princípio de segregação de interfaces, podemos dividir essa interface em partes menores, como ProcessarPagamentoPix, ProcessarPagamentoCartao e ProcessarPagamentoBoleto, garantindo que cada classe implementa apenas a interface relevante à sua função específica.

Relação entre acoplamento e coesão

A relação entre acoplamento e coesão é um ponto-chave no design de software. O cenário ideal é aquele onde conseguimos manter um baixo acoplamento entre os componentes e, ao mesmo tempo, garantir uma alta coesão dentro de cada módulo ou classe. No entanto, é necessário ter cautela, pois ao tentar otimizar um desses aspectos, podemos afetar negativamente o outro.

A Busca pelo Equilíbrio

A coesão reflete o grau de foco de uma classe ou módulo. Quanto mais os métodos e atributos de uma classe estiverem relacionados entre si, maior

será sua coesão. Já o acoplamento mede o nível de dependência entre diferentes classes ou módulos. O objetivo é manter essa dependência o mais fraca possível, ou seja, reduzir o acoplamento, para que uma alteração em um componente não exija mudanças em vários outros.

O desafio é que, ao aumentar a coesão em uma classe, corremos o risco de aumentar o acoplamento com outras classes. Da mesma forma, ao reduzir o acoplamento, podemos acabar fragmentando funcionalidades que, antes, eram altamente coesas. A chave está em encontrar um equilíbrio entre esses dois conceitos.

Exemplo Prático

Vamos observar um exemplo prático. Imagine a classe Pedido, que possui três métodos: `CalcularTotal()`, `Validar()` e `ProcessarPagamento()`. Esses métodos estão relacionados às responsabilidades de um pedido, o que confere à classe uma alta coesão, pois tudo está diretamente conectado ao contexto de um pedido.

No entanto, essa estrutura apresenta um alto acoplamento. Se houver uma mudança nas formas de pagamento, como a inclusão de um novo método de pagamento, seria necessário alterar o código da classe Pedido diretamente. Isso indica que o método `ProcessarPagamento()` está fortemente acoplado às outras funcionalidades da classe.

Para reduzir o acoplamento, podemos separar essas responsabilidades em diferentes classes: `CalculadorDeTotal`, `ValidadorDePedido` e `ProcessadorDePagamento`. Agora, cada funcionalidade está isolada em sua própria classe, o que reduz o acoplamento. No entanto, a coesão da classe Pedido foi diminuída, pois ela não está mais diretamente ligada às ações que envolvem um pedido. A classe Pedido se torna apenas um “orquestrador” que coordena a interação entre as outras classes.

Outro Exemplo: Autenticação, Autorização e Registro

Vamos considerar outro caso com as classes Autenticação, Autorização e Registro, que estão separadas e possuem um baixo acoplamento. Uma classe não depende da outra, e cada uma realiza sua função de maneira isolada. No

entanto, com essa separação, a coesão também foi prejudicada, pois essas funcionalidades estão todas relacionadas ao gerenciamento de usuários.

Para melhorar a coesão, podemos agrupar essas funções em uma única classe, como `GestorDeUsuário`, que lida com a autenticação, autorização e registro de usuários. Agora, temos uma alta coesão, pois todas essas funções estão intimamente relacionadas. Contudo, isso aumenta o acoplamento, já que qualquer mudança em uma das funções, como a autenticação, pode exigir alterações em toda a classe.

O design de software envolve constantemente a busca pelo equilíbrio entre acoplamento e coesão. Nem sempre é possível maximizar ambos simultaneamente. A prática recomendada é avaliar cada caso cuidadosamente, evitando que a redução de um afete negativamente o outro, e sempre buscando o melhor compromisso para a manutenção e evolução do sistema.

Acoplamento e coesão nos diagramas do Astah

Nesta seção, a ideia é compreender como esses princípios podem ser visualizados e ajustados diretamente nos diagramas, promovendo um design mais eficiente.

Trabalhando com Acoplamento e Coesão no Astah

Ao criar um diagrama de classes no Astah, podemos identificar visualmente tanto o acoplamento quanto a coesão de uma classe ou de um sistema. Vamos começar criando uma classe chamada `Pedido` no diagrama. Esta classe conterá os métodos `CalcularTotal()`, `Validar()` e `ProcessarPagamento()`. Nesse cenário, observamos que a classe tem alta coesão, pois todos os métodos estão relacionados ao processamento de pedidos. No entanto, também detectamos um alto acoplamento, já que qualquer mudança no processamento de pagamento, por exemplo, exigiria alterações diretamente na classe `Pedido`.

Refatoração para Reduzir o Acoplamento

Para resolver o problema do acoplamento, podemos refatorar a classe Pedido. Criamos, então, novas classes: CalculadorDeTotal, ValidadorDePedido e ProcessadorDePagamento, movendo as funcionalidades de cálculo, validação e processamento para essas classes especializadas. Com essa refatoração, conseguimos reduzir o acoplamento, pois cada funcionalidade está separada e independente. Se surgir uma nova forma de pagamento, podemos facilmente adicionar uma nova classe sem alterar as existentes.

No entanto, ao refatorar dessa forma, a coesão da classe Pedido foi comprometida. Agora, a classe Pedido atua apenas como um orquestrador das outras classes, e não mais como o núcleo das operações relacionadas ao pedido.

Exemplo de Diagrama com Baixa Coesão

Agora, vamos criar um novo exemplo no Astah para observar como o acoplamento e a coesão se comportam. Vamos criar três classes separadas: GerenciadorHotel, CalculadoraContas e EnviadorConfirmacoes. Cada uma dessas classes tem um método responsável por uma função específica: fazerReserva(), calcularConta() e enviarConfirmacao(). Embora esse design apresente baixo acoplamento, já que cada classe é independente, a coesão também é baixa, pois essas classes não estão focadas em uma responsabilidade unificada.

Refatoração para Aumentar a Coesão

Para melhorar a coesão, podemos agrupar essas funções em uma única classe chamada GerenciadorHotel, que incluirá os métodos fazerReserva(), calcularConta() e enviarConfirmacao(). Dessa forma, conseguimos uma alta coesão, pois todas as funcionalidades relacionadas à gestão de hotel estão em um único lugar. No entanto, isso resulta em alto acoplamento, já que qualquer mudança em uma das funções pode afetar toda a classe.

O Equilíbrio entre Acoplamento e Coesão

Ao longo do processo de design no Astah, é fundamental buscar o equilíbrio entre coesão e acoplamento. Aumentar a coesão pode gerar acoplamento

excessivo, e reduzir o acoplamento pode enfraquecer a coesão. O uso correto das ferramentas de diagramação permite visualizar e ajustar essas características, promovendo um design mais eficiente e flexível.

Conteúdo Bônus

Para se aprofundar no tema desta aula, sugiro que assista ao vídeo intitulado “Aula sobre visão geral de coesão e acoplamento - disciplina de POOI4 do curso ADS”, que está disponível no canal Andre Constantino no YouTube.

Referência Bibliográfica

ASCENCIO, A. F. G.; CAMPOS, E. A. V. de. Fundamentos da programação de computadores. 3.ed. Pearson: 2012.

BRAGA, P. H. Teste de software. Pearson: 2016.

GALLOTTI, G. M. A. (Org.). Arquitetura de software. Pearson: 2017.

GALLOTTI, G. M. A. Qualidade de software. Pearson: 2015.

MEDEIROS, E. Desenvolvendo software com UML 2.0 definitivo. Pearson: 2004.

MORAIS, I. S. de. (Org.). Engenharia de software. Pearson: 2017.

PFLEEGER, S. L. Engenharia de software: teoria e prática. 2.ed. Pearson: 2003.

SOMMERVILLE, I. Engenharia de software. 10.ed. Pearson: 2019